```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
df = pd.read_csv("loan_data (1).csv")

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
```

```
Dataset loaded successfully!
Shape of dataset: (307511, 122)
```

```
df.head()
```

	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	AMT_CR
	Cash loans	M	N	Y	0	202500.0	406
	Cash loans	F	N	N	0	270000.0	1293
	Revolving loans	M	Y	Y	0	67500.0	135
	Cash loans	F	N	Y	0	135000.0	312
	Cash loans	M	N	Y	0	121500.0	513

```
# Check the number of null values in each column

null_values = df.isnull().sum()

print(null_values)
```

```
SK_ID_CURR                      0
TARGET                          0
NAME_CONTRACT_TYPE              0
CODE_GENDER                     0
FLAG_OWN_CAR                    0
                             ...
AMT_REQ_CREDIT_BUREAU_DAY     41519
AMT_REQ_CREDIT_BUREAU_WEEK    41519
AMT_REQ_CREDIT_BUREAU_MON     41519
AMT_REQ_CREDIT_BUREAU_QRT     41519
AMT_REQ_CREDIT_BUREAU_YEAR    41519
Length: 122, dtype: int64
```

```
# Calculate the percentage distribution of TARGET

target_percentage = df['TARGET'].value_counts(normalize=True) * 100

print("Percentage distribution of TARGET:")
print(target_percentage)
```

```
Percentage distribution of TARGET:
TARGET
0    91.927118
1     8.072882
Name: proportion, dtype: float64
```

```
from sklearn.model_selection import train_test_split

X = df.drop('TARGET', axis=1)
y = df['TARGET']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (246008, 121)
Testing data: (61503, 121)
```

```
print("Before balancing:")
print(y_train.value_counts())
```

```
Before balancing:
TARGET
0    226148
1     19860
Name: count, dtype: int64
```

```
from sklearn.utils import resample

# Combine training features and target
train_data = pd.concat([X_train, y_train], axis=1)

# Separate majority and minority classes
majority = train_data[train_data['TARGET'] == 0]
minority = train_data[train_data['TARGET'] == 1]

print("Majority class:", len(majority))
print("Minority class:", len(minority))
```

```
Majority class: 226148
Minority class: 19860
```

```
majority_downsampled = resample(
    majority,
    replace=False,
    n_samples=len(minority),
    random_state=42
)
```

```
balanced_train = pd.concat([
    majority_downsampled,
    minority
])

# Shuffle the balanced dataset
balanced_train = balanced_train.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

print("Balanced training data:", balanced_train.shape)
```

```
Balanced training data: (39720, 122)
```

```
X_train_balanced = balanced_train.drop('TARGET', axis=1)
y_train_balanced = balanced_train['TARGET']
```

```
print(y_train_balanced.value_counts())
```

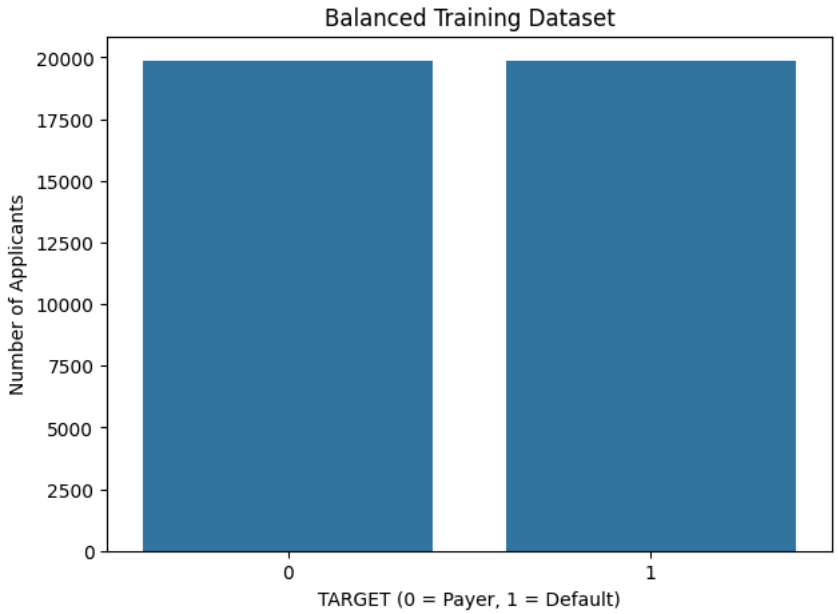
```
TARGET
1    19860
0    19860
Name: count, dtype: int64
```

```
plt.figure(figsize=(7,5))

sns.countplot(x=y_train_balanced)

plt.title('Balanced Training Dataset')
plt.xlabel('TARGET (0 = Payer, 1 = Default)')
plt.ylabel('Number of Applicants')

plt.show()
```




```
# Identify categorical columns
cat_cols = X_train_balanced.select_dtypes(include=['object']).columns.tolist()
print(f"Found {len(cat_cols)} categorical columns:")
print(cat_cols)
```

```
Found 16 categorical columns:
['NAME_CONTRACT_TYPE', 'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_TYPE_SUITE', 'NAME_INCOME_TYPE', 'NAME_EDUCAT
```

```
# Combine train and test temporarily to align columns during One-Hot Encoding
all_features = pd.concat([X_train_balanced, X_test], axis=0, keys=['train', 'test'])

# Perform One-Hot Encoding on all categorical columns
all_features_encoded = pd.get_dummies(all_features, columns=cat_cols, drop_first=True)

# Separate back into train and test sets
X_train_encoded = all_features_encoded.xs('train')
X_test_encoded = all_features_encoded.xs('test')

print("Shape of X_train_encoded:", X_train_encoded.shape)
print("Shape of X_test_encoded:", X_test_encoded.shape)
display(X_train_encoded.head())
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import recall_score, classification_report, confusion_matrix, accuracy_score
import numpy as np

# Handle potential missing values (NaNs) in the encoded datasets by simple imputation
# (Some columns may have NaNs which would cause scikit-learn models to fail)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')

X_train_imputed = imputer.fit_transform(X_train_encoded)
X_test_imputed = imputer.transform(X_test_encoded)

# Initialize and train a Random Forest classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train_imputed, y_train_balanced)
```

```
# Make predictions on the test set
y_pred = rf_model.predict(X_test_imputed)

# Calculate Sensitivity (Recall for class 1)
sensitivity = recall_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Sensitivity (Recall for Default Class 1): {sensitivity:.4f}\n")

# Print Confusion Matrix and full Classification Report
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.6950
Sensitivity (Recall for Default Class 1): 0.6657

Confusion Matrix:
[[39440 17098]
 [ 1660  3305]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.70      0.81     56538
           1       0.16      0.67      0.26      4965

    accuracy                           0.70     61503
   macro avg       0.56      0.68      0.53     61503
weighted avg       0.90      0.70      0.76     61503
```

```
from sklearn.metrics import roc_auc_score, roc_curve

# Get predicted probabilities for the positive class (Class 1)
y_pred_proba = rf_model.predict_proba(X_test_imputed)[:, 1]

# Calculate the ROC AUC Score
roc_auc = roc_auc_score(y_test, y_pred_proba)
print(f"ROC AUC Score: {roc_auc:.4f}")

# Compute ROC curve values
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

# Plot the ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR / Sensitivity)')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

```
ROC AUC Score: 0.7373
```

